

Language Models

Technical Notes

Rodrigo Kang

Table of contents

1	Language as a Representation Problem	1
2	Bag-of-Words	2
2.1	Tokenisation	3
2.2	Vocabulary	4
2.3	Vector Representation	4
2.4	Algorithm	6
2.5	Python Implementation	7

1 Language as a Representation Problem

Human language is naturally expressed through discrete symbols such as words, subwords, and characters. A machine-learning model, however, does not operate directly on these symbols. Its inputs must ultimately be represented numerically.

This creates a fundamental problem in language modelling:

How can language be represented numerically while preserving the information required to perform a particular task?

Consider, for example, the following two sentences:

The cat is sleeping.

The dog is sleeping.

To a human reader, the sentences are clearly related. They have almost the same syntactic structure and describe similar situations, differing primarily in the subject. A computational model, however, cannot directly perform mathematical operations on the words *cat*, *dog*, or *sleeping*. We first need a mapping from language to a numerical representation.

Let a text sequence be denoted by

$$S = (w_1, w_2, \dots, w_N), \quad (1)$$

where each w_i represents a linguistic unit, such as a word or token. We seek a representation function

$$\varphi : \mathcal{S} \rightarrow \mathbb{R}^d, \quad (2)$$

which maps a sequence S from the space of possible text sequences $\mathcal{S}$ to a numerical vector

$$\mathbf{x} = \varphi(S) = (x_1, x_2, \dots, x_d)^\top. \quad (3)$$

Once language has been represented numerically, the resulting vector can be used as the input to computational methods for tasks such as classification, retrieval, clustering, or generation.

The difficulty is that there is no unique numerical representation of language. Different mappings preserve different kinds of information. For example, a representation based only on word counts can preserve information about which words occur in a document and how frequently they occur, but it largely discards word order and context. More sophisticated representations attempt to preserve semantic or contextual relationships between linguistic units.

The representation therefore determines, to a significant extent, what information is available to the model. This observation provides a useful way to understand the historical development of language models. Many important advances in Natural Language Processing can be viewed as increasingly expressive ways of representing language:

$$\text{word counts} \longrightarrow \text{static embeddings} \longrightarrow \text{contextual representations}.$$

We begin with one of the simplest numerical representations of text: the **bag-of-words model**.

2 Bag-of-Words

The representation problem becomes concrete as soon as we try to perform a computational task with text. Suppose, for example, that we have collected three short customer reviews:

The product is good.

The product is very good.

The service is bad.

Before these reviews can be used by a machine-learning algorithm, we need to transform them into numerical representations. One of the simplest approaches is the **bag-of-words model**.

Bag-of-words represents a document by the words that occur in it and their frequencies. The central idea is deliberately simple: construct a vocabulary from the available documents and associate each vocabulary term with one component of a vector.

2.1 Tokenisation

Let the collection of documents, or **corpus**, be

$$\mathcal{D} = \{D_1, D_2, \dots, D_M\}, \quad (4)$$

where M is the number of documents.

For our example,

$$\begin{aligned} D_1 &= \text{"the product is good"}, \\ D_2 &= \text{"the product is very good"}, \\ D_3 &= \text{"the service is bad"}. \end{aligned}$$

The first step is **tokenisation**, which divides each document into smaller linguistic units called tokens. For now, we use the simplest possible tokenizer: convert the text to lowercase, remove punctuation, and split on whitespace.

This gives

$$\begin{aligned} T(D_1) &= (\text{the}, \text{product}, \text{is}, \text{good}), \\ T(D_2) &= (\text{the}, \text{product}, \text{is}, \text{very}, \text{good}), \\ T(D_3) &= (\text{the}, \text{service}, \text{is}, \text{bad}). \end{aligned}$$

This simple procedure is sufficient for the present example, but tokenisation is itself a nontrivial problem. Modern language models commonly operate on subword or byte-level tokens rather than simply splitting text on whitespace. Tokenisation will therefore be studied separately later.

2.2 Vocabulary

Once the documents have been tokenised, we construct a **vocabulary** containing the unique tokens appearing in the corpus.

For the example above,

$$\mathcal{V} = \{\text{the, product, is, good, very, service, bad}\}.$$

The vocabulary therefore contains

$$|\mathcal{V}| = 7$$

distinct terms.

Let

$$\mathcal{V} = (v_1, v_2, \dots, v_d), \tag{5}$$

where

$$d = |\mathcal{V}|.$$

The ordering of the vocabulary is arbitrary, but once an ordering has been chosen it must remain fixed because each term determines the meaning of one coordinate of the resulting vectors.

For our chosen ordering,

$$\begin{aligned} v_1 &= \text{the}, & v_2 &= \text{product}, & v_3 &= \text{is}, \\ v_4 &= \text{good}, & v_5 &= \text{very}, & v_6 &= \text{service}, & v_7 &= \text{bad}. \end{aligned}$$

2.3 Vector Representation

We can now define the bag-of-words representation of a document D as

$$\varphi(D) = \mathbf{x} = (x_1, x_2, \dots, x_d)^\top, \tag{6}$$

where each component is the number of times the corresponding vocabulary term occurs in the document:

$$x_j = \text{count}(v_j, D), \quad j = 1, \dots, d. \quad (7)$$

Consider the first document,

$$D_1 = \text{"the product is good"}.$$

Using the vocabulary defined above, the individual counts are

$$\begin{aligned} \text{count}(\text{the}, D_1) &= 1, \\ \text{count}(\text{product}, D_1) &= 1, \\ \text{count}(\text{is}, D_1) &= 1, \\ \text{count}(\text{good}, D_1) &= 1, \\ \text{count}(\text{very}, D_1) &= 0, \\ \text{count}(\text{service}, D_1) &= 0, \\ \text{count}(\text{bad}, D_1) &= 0. \end{aligned}$$

Therefore,

$$\varphi(D_1) = \begin{pmatrix} 1 \\ 1 \\ 1 \\ 1 \\ 0 \\ 0 \\ 0 \end{pmatrix}.$$

Applying the same procedure to the remaining documents gives

$$\varphi(D_2) = \begin{pmatrix} 1 \\ 1 \\ 1 \\ 1 \\ 1 \\ 0 \\ 0 \end{pmatrix}, \quad \varphi(D_3) = \begin{pmatrix} 1 \\ 0 \\ 1 \\ 0 \\ 0 \\ 1 \\ 1 \end{pmatrix}.$$

The complete corpus can therefore be represented by the **document-term matrix**

$$X = \begin{pmatrix} 1 & 1 & 1 & 1 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 & 1 & 1 \end{pmatrix}, \quad (8)$$

where each row represents a document and each column represents a term in the vocabulary.

More generally,

$$X \in \mathbb{R}^{M \times d},$$

with elements

$$X_{ij} = \text{count}(v_j, D_i). \quad (9)$$

Bag-of-words has therefore transformed a collection of variable-length symbolic sequences into fixed-dimensional numerical vectors that can be processed by standard statistical and machine-learning algorithms.

2.4 Algorithm

The construction can be expressed as the following procedure:

Input: a corpus $\mathcal{D} = \{D_1, \dots, D_M\}$.

Output: a vocabulary $\mathcal{V}$ and document-term matrix X .

1. Tokenise every document in the corpus.
2. Collect the unique tokens across all documents to construct the vocabulary $\mathcal{V}$.
3. Assign each vocabulary term v_j to a column j .
4. Initialise $X \in \mathbb{R}^{M \times d}$ with zeros.
5. For each document D_i :
 1. For each token in D_i :
 1. Find the vocabulary index j associated with the token.
 2. Increment X_{ij} by one.
6. Return $\mathcal{V}$ and X .

The computational representation is now explicit: text has become a matrix of numerical features. However, this simplicity comes at a cost. The representation records **which words occur and how often they occur**, but not the order in which they occur.

For example, the sentences

dog bites man

and

man bites dog

contain exactly the same tokens with exactly the same frequencies. Under a unigram bag-of-words representation, both therefore produce the same vector:

$$\varphi(\text{"dog bites man"}) = \varphi(\text{"man bites dog"}).$$

Their meanings are clearly different, but that difference has been lost during the mapping from language to the numerical representation.

This illustrates a fundamental trade-off. Bag-of-words provides a simple and useful numerical representation of text, but it does not model word order, context, or semantic relationships between words. These limitations motivate the development of richer language representations.

2.5 Python Implementation

The bag-of-words representation can be implemented directly from the procedure described above. To make the construction explicit, the first implementation uses only Python's built-in data structures rather than a specialised Natural Language Processing or machine-learning library.

The implementation first tokenises each document, constructs the vocabulary from the unique tokens in the corpus, and then counts the occurrences of each vocabulary term in every document. The resulting vectors are collected into the document-term matrix X .

The complete implementation is available in `python/bag_of_words.py`.

For the corpus used in the previous example,

$$\begin{aligned} D_1 &= \text{"the product is good"}, \\ D_2 &= \text{"the product is very good"}, \\ D_3 &= \text{"the service is bad"}, \end{aligned}$$

the implementation produces the vocabulary

$$\mathcal{V} = (\text{the, product, is, good, very, service, bad}),$$

and the document-term matrix

$$X = \begin{pmatrix} 1 & 1 & 1 & 1 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 & 1 & 1 \end{pmatrix}.$$

The numerical result therefore reproduces the representation obtained analytically.